UNIVERSIDAD AMERICANA · UAM

EVIDENCIAS Y ANÁLISIS

Laboratorio de concurrencia en Java y JavaFX

Predicción, ejecución y análisis de las cinco estaciones, capturas de la aplicación, tabla de pruebas y retos de análisis colaborativo.

Asignatura	CNU0525 · Programación de Aplicaciones de Escritorio
Grupo	1
Docente	José Durán García
Integrantes	Daniella Rocha Kellys Bellanger Nora Obregón Daris Mayorga
Repositorio	github.com/kellys-bellanger/Laboratorio_Hilos
Fecha	26 de octubre de 2026

CNU0525 · PROGRAMACIÓN DE APLICACIONES DE ESCRITORIO · GRUPO 1

Evidencias y análisis: Laboratorio de concurrencia en Java y JavaFX

Profesor: José Durán García · **Repositorio:** github.com/kellys-bellanger/Laboratorio_Hilos

Regla del laboratorio: *primero prediga, después ejecute, observe y explique*. Cada estación sigue esas cuatro partes. La salida completa de consola se entrega en la carpeta `evidencias` y las capturas en `capturas`.

Equipo y roles

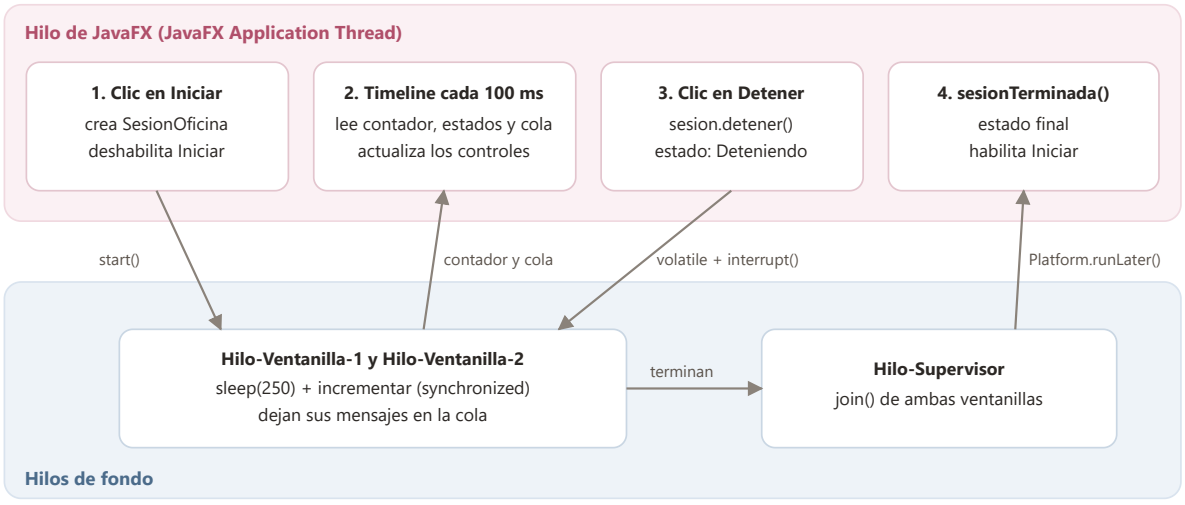
Integrante	Rol	Aporte principal	Rama
Daniella Rocha	**Líder**: revisa los PR e integra en `main`	Estación 5 (integración JavaFX), integración de ramas, pruebas JUnit, documentación	`feature/daniella`
Kellys Bellanger	Integrante 1	Base del proyecto, Estación 1 (Thread y Runnable)	`feature/kellys`
Nora Obregón	Integrante 2	Estación 2 (estados y monitor) y Estación 4 (ventanilla cancelable)	`feature/nora`
Daris Mayorga	Integrante 3	Estación 3 (contador compartido, `synchronized`, `AtomicInteger`)	`feature/daris`

Ramas en GitHub: `main`, `feature/daniella`, `feature/kellys`, `feature/nora` y `feature/daris`.

Preparación del entorno

- **Java:** OpenJDK 21.0.12 (Eclipse Temurin). **Maven:** 3.8.5 mediante el wrapper `mvnw`.
- **JavaFX 21.0.6** se descarga como dependencia de Maven (`javafx-controls`). El proyecto es modular (`module-info.java` con `requires javafx.controls`), así que JavaFX se carga desde el *module path*. Se ejecuta con `mvnw javafx:run` o desde IntelliJ con la clase `Launcher`.
- **Paquetes:** `app` (ventana y pruebas de consola), `tareas` (lógica de hilos) y `modelo` (`EstadoVentanilla`).

Flujo de la aplicación



Operaciones del hilo de JavaFX: responder a los clics, crear la sesión, habilitar o deshabilitar botones y modificar cualquier control (etiquetas, barra de progreso, registro). **Operaciones de fondo:** atender solicitudes (`sleep`), incrementar el contador y esperar con `join()` .

Estación 1. Thread y Runnable

Predicción Si el controlador invoca una tarea que duerme 500 ms cinco veces, la interfaz se congela 2,5 s, porque el `sleep()` ocurre en el mismo hilo que dibuja la ventana y atiende los clics. En un hilo independiente la ventana sigue respondiendo, porque el hilo de JavaFX queda libre.

Ejecución `PruebaEstacion1` llama primero a `run()` y luego a `start()` sobre dos `Thread` llamados `Hilo-Ventanilla-1` y `Hilo-Ventanilla-2` . Se ejecutó tres veces:

Ejecución	Con `run()`	Primeros mensajes con `start()`
1	Todo en `[main]` : Ventanilla 1 (1 a 5) y luego Ventanilla 2 (1 a 5)	V2-1, V1-1, `[main]` termina, V1-2, V2-2, V1-3
2	Igual que la ejecución 1	V1-1, V2-1, `[main]` termina, V2-2, V1-2, V1-3
3	Igual que la ejecución 1	V2-1, V1-1, `[main]` termina, V2-2, V1-2, V1-3

```
[Hilo-Ventanilla-2] - Ventanilla 2 procesa solicitud 1
[Hilo-Ventanilla-1] - Ventanilla 1 procesa solicitud 1
[main] El método main terminó de invocar start().
[Hilo-Ventanilla-1] - Ventanilla 1 procesa solicitud 2
```

Análisis

- `run()` es una llamada normal a un método: no crea hilos, todo corre en `main` y en orden.
- Con `start()` el orden cambia entre ejecuciones porque el **planificador del sistema operativo** decide qué hilo corre en cada momento; no hay un orden garantizado.
- **No** es correcto asumir que `start()` termina el trabajo antes de devolver el control: en las tres ejecuciones `main` imprimió que terminó de invocar `start()` cuando los hilos apenas habían hecho su primera solicitud. `start()` solo pide crear el hilo y regresa de inmediato.

Estación 2. Estados del hilo

Estado	Significado	¿Observado?
`NEW`	El `Thread` fue creado pero no se ha llamado a `start()`	Sí, en consola y en una prueba JUnit
`RUNNABLE`	Se está ejecutando o está listo para ejecutarse	Sí, al iniciar la ventanilla
`BLOCKED`	Espera entrar a un bloque `synchronized` que otro hilo tiene tomado	No; ocurriría por instantes cuando dos hilos compiten por el contador
`WAITING`	Espera sin límite de tiempo (`join()`, `wait()`)	No se capturó; es el estado de `Hilo-Supervisor` durante su `join()`
`TIMED_WAITING`	Espera con límite de tiempo (`sleep(ms)`, `join(ms)`)	Sí, durante el `sleep()` de cada solicitud
`TERMINATED`	El método `run()` terminó	Sí, al detener o finalizar

Predicción Como cada solicitud duerme casi todo el tiempo, el monitor verá sobre todo `TIMED_WAITING`, y `RUNNABLE` solo por instantes.

Ejecución `PruebaParte2` usa `MonitorEstados`, un hilo aparte que consulta `getState()` cada 300 ms:

```
[EVIDENCIA] Estado justo al instanciar (NEW): NEW
[MONITOR] Estado de 'Hilo-Ventanilla-1': RUNNABLE
[MONITOR] Estado de 'Hilo-Ventanilla-1': TIMED_WAITING
[MONITOR] Estado de 'Hilo-Ventanilla-1': TIMED_WAITING
...
MONITOR: Estado final de 'Hilo-Ventanilla-1': TERMINATED
```

En la ventana, el monitor no bloquea el hilo gráfico porque es un `Timeline`: las tarjetas muestran `TIMED_WAITING` mientras se atiende y `TERMINATED` al terminar.

Experimento Para observar `TIMED_WAITING`, la tarea duerme (`sleep(1000)` en `VentanillaCancelable`, `sleep(250)` en `VentanillaOficina`) y el estado se consulta con un intervalo menor que el sueño: la mayoría de las lecturas cae dentro del `sleep()`.

Razonamiento `getState()` devuelve el estado **en el instante de la consulta**. El hilo sigue corriendo en paralelo y puede cambiar justo después, así que el valor puede estar desactualizado cuando se imprime. Sirve para observar y depurar, no para decidir; para saber que un hilo terminó se usa `join()`.

Estación 3. Sincronización de recursos

Predicción Sin protección algunos incrementos se perderán y el total será menor que el esperado; con `synchronized` o `AtomicInteger` siempre será exacto.

Ejecución `PruebaConsola` lanza dos hilos, espera con `join()` (en consola, nunca en el hilo de JavaFX) y repite 10 veces cada variante.

Variante	1 000 por hilo (esperado 2 000)	100 000 por hilo (esperado 200 000)
Sin protección	**1 de 10** con pérdidas (1 540)	**3 de 10** con pérdidas (142 109, 199 878, 188 217)
`synchronized`	0 de 10	0 de 10
`AtomicInteger`	0 de 10	0 de 10

```
--- PRUEBA 1: SIN PROTECCIÓN (Esperado: 200000) ---
Run  1: Total = 142109 <- ¡Condición de Carrera! (57891 perdidos)
Run  2: Total = 199878 <- ¡Condición de Carrera! (122 perdidos)
Run  3: Total = 188217 <- ¡Condición de Carrera! (11783 perdidos)
Run  4: Total = 200000
...
Corridas con pérdidas: 3 de 10
```

> Con 1 000 la carrera casi no aparece porque cada hilo termina en microsegundos, a veces antes de que arranque el otro. Por eso `PruebaConsola` acepta la cantidad como argumento y por defecto usa 100 000.

¿Qué cambió y por qué? `total++` son tres pasos: leer, sumar y escribir. Sin protección, dos hilos pueden leer el mismo valor y escribir el mismo resultado, y un incremento se pierde. `synchronized` permite que solo un hilo a la vez ejecute el método (exclusión mutua) y garantiza que el siguiente hilo vea el valor actualizado (visibilidad).

Desafío Se hicieron las dos alternativas. La **lectura segura** (`getTotalSincronizado()` también es `synchronized`) asegura leer el último valor escrito. `AtomicInteger` incrementa de forma atómica sin bloquear (*compare-and-swap*); es más simple y rápido para un solo contador, mientras que `synchronized` sirve para proteger varias operaciones juntas.

synchronized frente a runLater Los controles de JavaFX no son seguros para hilos y **solo** se pueden modificar desde el *JavaFX Application Thread*; si otro hilo lo intenta, JavaFX lanza `IllegalStateException: Not on FX application thread` o deja la vista inconsistente. `synchronized` solo evita que dos hilos entren a la vez a un bloque, pero el código sigue corriendo en el hilo de fondo. `Platform.runLater()` es lo que **mueve** el código al hilo de JavaFX.

Estación 4. Detención y control

Predicción Si se interrumpe a la ventanilla mientras duerme, `sleep()` lanzará `InterruptedException` de inmediato, sin esperar a que termine la pausa.

Ejecución `VentanillaOficina` (y `VentanillaCancelable` en consola) usa una bandera `volatile boolean activo` más `interrupt()`. El estado visible pasa por En espera → Ejecutando → Deteniendo → Detenido o Finalizado.

```
[Hilo-Ventanilla-2] Ventanilla 2 fue interrumpida durante sleep().
[Hilo-Ventanilla-1] Ventanilla 1 fue interrumpida durante sleep().
[Hilo-Ventanilla-1] Ventanilla 1 terminó en estado Detenido (18 de 20 solicitudes).
[Hilo-Ventanilla-2] Ventanilla 2 terminó en estado Detenido (18 de 20 solicitudes).
--- Sesión terminada: 36 de 40 solicitudes. Hilos vivos: no ---
```

La prueba JUnit `detenerInterrumpeElSleepYTerminaDetenida` comprueba que tras `detener()` el hilo termina en menos de 250 ms, es decir, sin esperar a que acabe el `sleep()`.

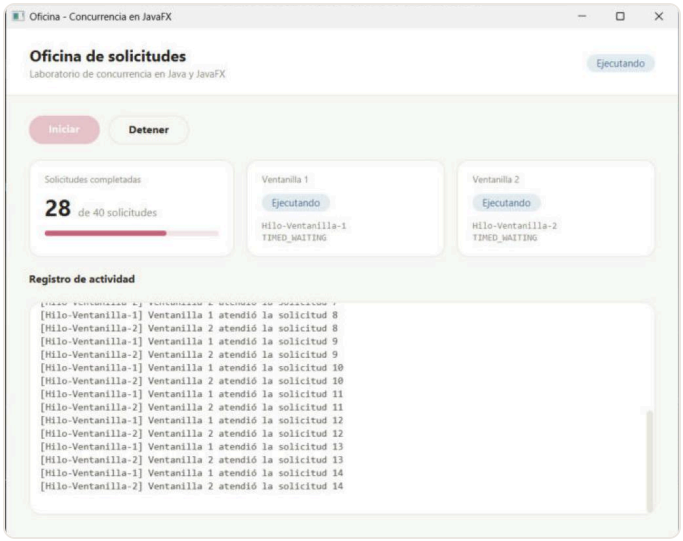
Análisis

- **Solicitar la interrupción no garantiza que el hilo ya terminó.** `interrupt()` y la bandera son solo avisos; el hilo termina cuando revisa la bandera o cuando su `sleep()` lanza la excepción. Para confirmarlo hay que esperar con `join()` o revisar `isAlive()`. Por eso la ventana muestra *Deteniendo* hasta que `Hilo-Supervisor` confirma el fin, y solo entonces *Detenido*.
- **No se debe ignorar** `InterruptedException` porque al lanzarse se **borra** la marca de interrupción. Si se atrapa y no se hace nada, el hilo sigue trabajando como si nadie le hubiera pedido parar. Se restaura con `Thread.currentThread().interrupt()` y se sale del ciclo.

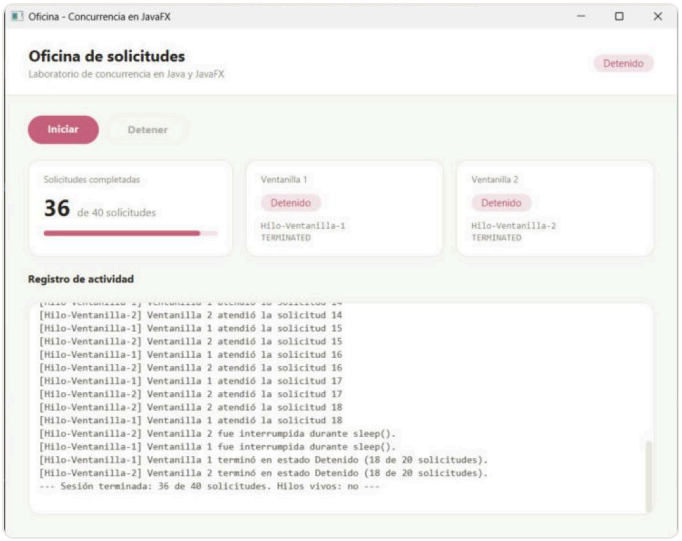
Estación 5. Integración JavaFX

Predicción Si el trabajo corre en hilos propios y la vista se actualiza en el hilo de JavaFX, la ventana responderá mientras las ventanillas trabajan.

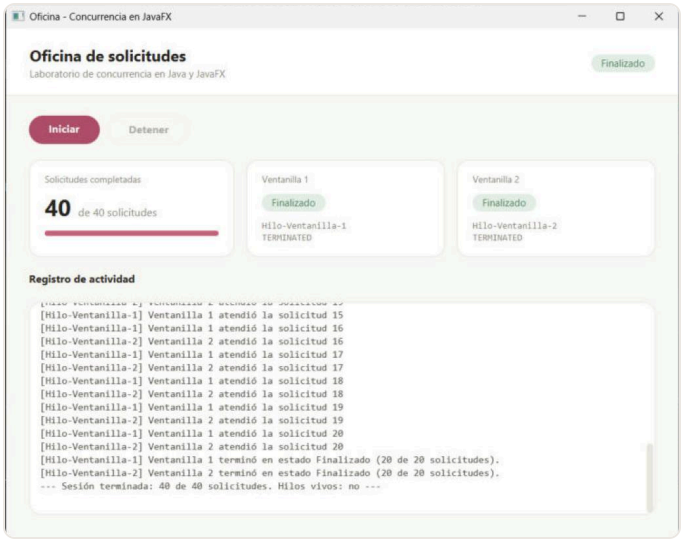
Ejecución La ventana tiene *Iniciar*, *Detener*, la etiqueta de estado, el contador con barra de progreso, una tarjeta por ventanilla y el registro de actividad.



Captura 1. Ejecutando: 28/40, ambas ventanillas en `TIMED_WAITING`, *Iniciar* deshabilitado.



Captura 2. Detenido: 36/40, interrupción durante `sleep()`, hilos `TERMINATED`, "Hilos vivos: no".



Captura 3. Nueva sesión finalizada: 40/40 sin pérdidas, ambas ventanillas en *Finalizado*.

Reto de diseño En lugar de un `Platform.runLater()` por mensaje, los hilos dejan sus mensajes en una `ConcurrentLinkedQueue` y un `Timeline` de 100 ms la vacía de una vez con un solo `appendText`. La interfaz se actualiza como máximo 10 veces por segundo, sin importar cuántos mensajes se generen. `Platform.runLater()` se usa una sola vez por sesión, para el aviso de fin.

Pruebas y evidencias

Prueba	Resultado esperado	Evidencia / hallazgo
Dos ventanillas	Ambas trabajan y la ventana responde	Captura 1: ambas en *Ejecutando*, el contador sube y la ventana responde; registro intercalado de los dos hilos.
Conteo concurrente	Total correcto sin pérdidas	Captura 3: 40/40. Consola: `synchronized` y `AtomicInteger` exactos en 40 corridas; sin protección hubo pérdidas. JUnit: 20 repeticiones con 200 000.
Cancelación	Las tareas finalizan cooperativamente	Captura 2: "fue interrumpida durante sleep()", estado *Detenido*, hilos `TERMINATED`. JUnit: `detenerTerminaAmbasVentanillasCooperativamente`.
Reinicio	No quedan hilos de la sesión anterior	Captura 2: "Hilos vivos: no" antes de habilitar *Iniciar*; la captura 3 es una sesión nueva desde 0. JUnit: `elAvisoDeFinSoloLlegaCuandoNoQuedanHilosVivos`.
Estados	Observaciones explicadas con evidencia	`NEW`, `RUNNABLE`, `TIMED_WAITING` y `TERMINATED` en consola y en las tarjetas; explicación en la Estación 2.

Pruebas automáticas: 32 pruebas JUnit 5 (`ContadorCompartidoTest`, `VentanillaOficinaTest`, `SesionOficinaTest`), todas aprobadas con `mvnw test`.

Organización con GitHub

- **Flujo:** cada integrante trabaja en su rama `feature/<nombre>` y abre un Pull Request hacia `main`; la líder lo revisa y lo integra.
- **Convención de commits:** prefijos `feat:`, `fix:`, `refactor:`, `style:`, `test:`, `docs:`, `chore:` y `build:` en modo imperativo. Algunos de los primeros commits no siguen bien la convención (incluyen `git commit -m` en el mensaje o usan `feat|`); no se reescribieron para no alterar un historial ya publicado, y los siguientes sí la siguen.
- **Archivos excluidos:** `.idea/` y `target/` están en `.gitignore`; se sacó del repositorio la carpeta `.idea` que se había subido por error.

Retos de análisis colaborativo

1. Riesgos si dos integrantes modifican el mismo controlador JavaFX a la vez

Conflictos de Git en el mismo archivo y, peor, cambios que se integran sin conflicto pero rompen la lógica del otro (un botón que deja de habilitarse, un `runLater` duplicado). Estrategia: separar la lógica de hilos (`tareas`) de la vista (`app`), asignar un responsable por archivo, hacer PR pequeños y actualizar la rama con `main` antes de abrir el PR.

2. ¿Por qué un PR que compila puede seguir siendo incorrecto en concurrencia?

El compilador no detecta condiciones de carrera, la falta de `volatile`, controles modificados fuera del hilo de JavaFX ni interrupciones ignoradas. Esos errores aparecen solo en algunas ejecuciones: con 1 000 incrementos la carrera apareció en 1 de 10 corridas. Por eso hacen falta pruebas repetidas, revisión de código y evidencia de ejecución.

3. Evidencias de que la detención fue controlada y la interfaz no se bloqueó

El registro muestra la interrupción durante `sleep()` y el estado final de cada ventanilla; las tarjetas muestran `TERMINATED`; aparece "Hilos vivos: no" antes de habilitar *Iniciar*; la ventana siguió respondiendo (el botón *Detener* funcionó a mitad de la sesión); y las pruebas JUnit de detención y reinicio pasan.

4. Resolver un conflicto de Git sin sobrescribir el trabajo de otro

1. `git fetch` y `git merge origin/main` en la rama propia (nunca `push --force`).
2. Abrir cada archivo en conflicto y leer las dos versiones entre `<<<<<<<` y `>>>>>>>`.
3. Combinar a mano conservando los cambios de ambos; si hay duda, preguntar al autor del otro cambio.
4. Compilar y ejecutar `mvnw test` antes de `git add` y `git commit`.
5. Subir los cambios y pedir revisión en el PR, indicando cómo se resolvió el conflicto.

En este proyecto la integración se hizo así: se fusionaron `feature/nora` y la rama de Daris en `feature/daniella`, se movieron las clases de Daris a su paquete y se corrigió `module-info`, sin borrar el trabajo de nadie; los commits originales conservan a su autor.